CSE486/586 Project

Andrew Dirr, AJ Marin, Dean DiCarlo

11 May 2026

1 Introduction

Traditional camera-based pose estimation works in optimal light settings, but can suffer under noisy and dark conditions. Supplementing with millimeter-wave radar can potentially help: although it does not capture visual features, it penetrates clothing and minor obstructions and operates in darkness. This project explores how well three deep learning architectures can recover 17 human-pose body joints from raw mmWave radar heatmaps, and whether learning biases such as specific attention to channels or dual-stream encoding result in significant differences from the performance of a standard baseline model.

2 Dataset

All experiments use the MMVR dataset (Protocol P1, Split S1), which pairs mmWave radar data with ground-truth keypoint annotations from an image capturing system. Each sample consists of a 2-channel radar tensor of shape (2, 256, 128). Channel 1 is the horizontal projection (azimuth–range), channel 2 is the vertical projection (elevation–range). Ground-truth 17 keypoint coordinates in normalized [0, 1] image space (640×480 reference frame) are accompanied by per-keypoint visibility flags. Ground-truth Gaussian heatmaps of size (17, 64, 64) are generated by placing a 2D Gaussian ($\sigma = 3$ pixels) centered at each visible keypoint. The P1S1 split was executed using a pre-assigned train/validation/test partition.

3 Model Architectures

All three models share the same input/output interface. The input is a tensor of shape $(B, 2, 256, 128)$ where B is the batch size, 2 is the number of channels, 256 the radar frame height and 128 the radar frame width. The output are Gaussian heatmaps $(B, 17, H, W)$ – where H and W represent height and width after application of convolutions – and 2D keypoint coordinates $(B, 17, 2)$, derived from the heatmaps using soft-argmax. All three models follow the same basic encoder-decoder structure (just with varying block arrangements and module additions): the encoder downsamples spatially using convolutions and the decoder upsamples followed by convolutions. They are all trained using the same loss function. Where the models are unique is explained below for each one.

3.1 ResNet-18 Baseline

ResNet-18 is a standard image classification network originally designed to take in a color photo using three RGB channels and output a class label. The basic idea of ResNet is *residual connection*: instead of each layer trying to learn the full transformation from input to output, it only learns the difference (the **res**idual) from its input. ResNet-18 has 18 layers of convolutions that are organized into four groups. Each one contains residual blocks. Here, we slightly repurpose ResNet18 to accept two radar channels instead of three RGB channels. We also drop layer 4 so the spatial dimensions are not as compressed and more information is retained before the decoder reconstructs to heatmaps. Weights are initialized as Kaiming uniform.

3.2 CustomCNN

The custom convolutional neural network model – customCNN model – has four stages ($32 \to 64 \to 128 \to 256$). Each one consists of a residual block followed by a convolutional block attention module (CBAM). The first stage looks at pixel patches that are 7×7. This allows the CNN to better capture broader, low-level spatial patterns in the beginning. Stages two, three and four contain a residual block (just like in ResNet-18) followed by a CBAM attention block. Adding CBAM is what makes this CNN different from ResNet. The CBAM applies channel attention ("Which feature channels are most import?") and spatial attention ("Which spatial locations in the feature map are most important?"), scaling channel and spatial locations accordingly. Rather than treating all channels and spatial positions equally, the network focuses on the most relevant features during the encoder stage before passing them on. This allows the customCNN model to concentrate on the most relevant parts of the radar signal. Weights for this model are initialized as Kaiming normal.

3.3 FusionModel

We were curious to see if treating the radar channels *separately* rather than stacking them together (like the other models do) made a difference. The idea was that each radar channel contains different information, and by letting encoders specialize on each one, they will produce richer features than encoders who try handling both at once. Thus, the FusionModel splits input tensors $(B, 2, 256, 128)$ into two separate $(B, 1, 256, 128)$ tensors, one per channel. HoriEncoder processes the horizontal channel, VertEncoder processes the vertical channel. Both have identical architectures (convultion $\to$ residual block $\to$ CBAM $\to$ residual block $\to$ CBAM). Each one produces a $(B, 128, 64, 32)$ feature map. They do not share the same weights, so they can learn different things. After separate horizontal and vertical encoding, the tensors are concatenated along the channel dimension into a single $(B, 256, 64, 32)$ tensor, combining what the parallel streams learned. A decoder then upsamples features to a higher resolution before heatmaps are produced and converted via soft-argmax to keypoint coordinates. Because we suspected the fusionModel would perform the best, we also exposed it during validation to more rigor than the other two models by adding validation segments that mimicked adversarial radar perturbations. These were Gaussian noise ($\sigma = 0.15$), drop-out (randomly zero-out 30% of radar pixels) and low-power (radar signal was scaled to only 20% of its amplitude). It should be noted that these additional validation modules for the fusionModel did not count toward its overall training time recordings. All weights throughout the fusionModel were initialized as Kaiming uniform.

4 Hyperparameter-Tuning, Training and Model Selection

All models were trained minimizing the loss function

$$\mathcal{L} = \lambda_{\text{hm}} \cdot \mathcal{L}_{\text{hm}} + \lambda_{\text{coord}} \cdot \mathcal{L}_{\text{coord}}$$

where λ_{hm} and λ_{coord} are regularization parameters for heatmap and coordinate-point loss, respectively. The loss function thus tracked how accurately the models' Gaussian heatmap and keypoint coordinate predictions matched ground truths. For the individual loss components, we have

$$\mathcal{L}_{\text{heatmap}} = \frac{1}{BKHW} \sum_{b=1}^{B} \sum_{k=1}^{K} \sum_{h=1}^{H} \sum_{w=1}^{W} \left(\sigma(\hat{M}_{bkhw}) - G_{bkhw} \right)^2$$

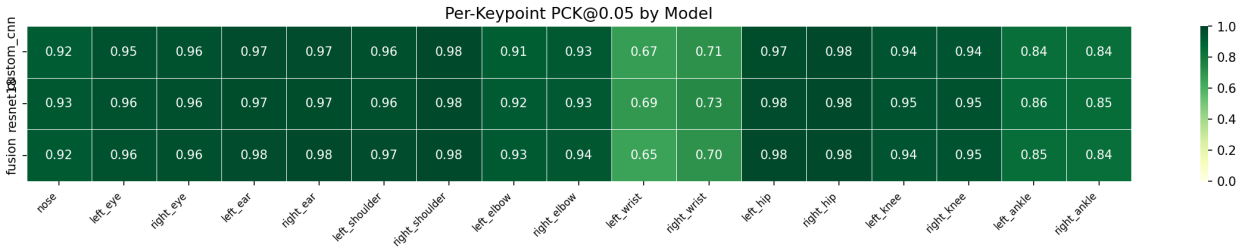

Figure 1: A matrix of percentage-of-correct-keypoint (PCK@0.05) values per keypoint and model. A predicted keypoint is correct if its Euclidean distance to the ground-truth is within 5% of the maximum image dimension (here this would be 5% of 640 pixels, i.e. 32 pixels). All models performed exceptionally well across nearly all keypoints with scores of around 90%. The most difficult body parts for the models to correctly identify were left and right wrists. Here scores were consistently around 70%.

where $\sigma(\hat{M}_{bkhw})$ is the sigmoid of the model logit at position (h, w) for keypoint k and batch b and G_{bkhw} is the ground-truth Gaussian heatmap value. Thus, $\mathcal{L}_{\text{heatmap}}$ is the mean squared error between the sigmoid-activated predicted heatmap and the Gaussian ground-truth heatmap. The other loss, $\mathcal{L}_{\text{coord}}$ is defined as

$$\mathcal{L}_{\text{coord}} = \frac{1}{N_{\text{vis}}} \sum_{b=1}^{B} \sum_{k=1}^{K} v_{bk} \cdot \Big(|\hat{x}_{bk} - x_{bk}| + |\hat{y}_{bk} - y_{bk}| \Big)$$

where $\hat{x}_{bk}$, $\hat{y}_{bk}$ are the predicted keypoint coordinates and x_{bk}, y_{bk} the true keypoint coordinates of keypoint k in sample b. The symbol v_{bk} represents the binary visibility flag, which is 1 if the keypoint is visible and 0 otherwise. N_{vis} is the total number of visible key points.

Hyperparameter tuning was carried out to determine optimal values for neural network weight decays λ_ω and learning rates η as well as λ_{hm} and λ_{coord} in the loss function. A grid of 240 possible combinations $(\lambda_\omega, \eta, \lambda_{\text{hm}}, \lambda_{\text{coord}})$ was generated. To save time, 30 grid points were randomly sampled and the models were trained on each one for 15 epochs (even with 8 processor cores and CUDA GPU support, this took over 40 hours). The final hyperparameter configuration determined by this grid search was $\lambda_\omega^* = 10^{-3}$, $\eta^* = 10^{-5}$, $\lambda_{\text{hm}}^* = 1$, $\lambda_{\text{coord}}^* = 0.5$.

Using these optimal hyperparameters, the three models were then trained for 30 epochs and validation losses were collected for a final performance assessment.

5 Results

All three models achieve strong performance in terms of purely statistical evaluation metrics. For PCK, the customCNN model achieves an average of 0.909, the ResNet-18 model an average score of 0.916 and the fusionModel an average of 0.912. For OKS, customCNN achieves a score of 0.958, ResNet-18 a score of 0.960 and the fusionModel a score of 0.959. Though just barely, the ResNet-18 baseline leads across all statistical metrics.

A closer per-keypoint analysis reveals a consistent pattern across all models in terms of their abilities to handle different body parts: upper-body joints (ears, shoulders, hips) exceed 97% PCK, while

Figure 2: Loss curves generated during training and validation of all three models using optimal hyperparameters as determined during tuning. The left shows the evolution of total loss $\mathcal{L}$ with each training/validation epoch, the right shows the evolution of the coordinate loss component $\mathcal{L}_{\text{coord}}$.

wrists achieve only 67–73% PCK — the lowest of all joints. This is consistent with wrists being the most mobile joints and most susceptible to blur and positional ambiguity.

While statistical performance scores are all extremely close, computation time and resource consumption should also be considered in the final evaluation of the models. It is safe to say that the fusionModel achieves the worst *overall* performance because it had nearly the same parameter count as ResNet-18, but its training times were far greater, averaging around one hour and fifty minutes per 15 epochs vs. thirteen to fifteen minutes for ResNet-18 and the customCNN model. From this, we can clearly see that, even though each model achieves more or less the same degree of statistical accuracy, they all do so at very different computational costs at the backend. The fact that the fusionModel's performance was more or less the same as the ResNet-18 baseline and the customCNN meant that its high computation costs were not justified.

6 Conclusion

We demonstrated that mmWave radar heatmaps contain sufficient spatial information for convolutional neural network architectures to localize 17 human body joints with over 90% PCK@0.05 accuracy. Surprisingly, the ResNet-18 baseline is the most effective model, achieving the best accuracy with the fewest training complications. While the customCNN and fusionModel offer lower parameter counts and achieve almost the same statistical performance, they do so at a much higher computation costs and still do not surpass the pretrained baseline.

The fact that ResNet18 came out on top tells us that the horizontal and vertical radar channels potentially contain redundant or weakly complementary information for the given task. The fusionModel was specifically designed to process the two radar views through separate encoders and then fuse them, the idea being that the two streams probably carry distinct spatial information worth preserving separately. If that bet paid off, the fusionModel should clearly win. A near-tie means either that the two radar views are already correlated enough that a single encoder treating them as a 2-channel input (ResNet-18) captures nearly everything that the dual-stream fusionModel does or the fusionModel isn't actually learning to cross-correlate the two streams meaningfully and

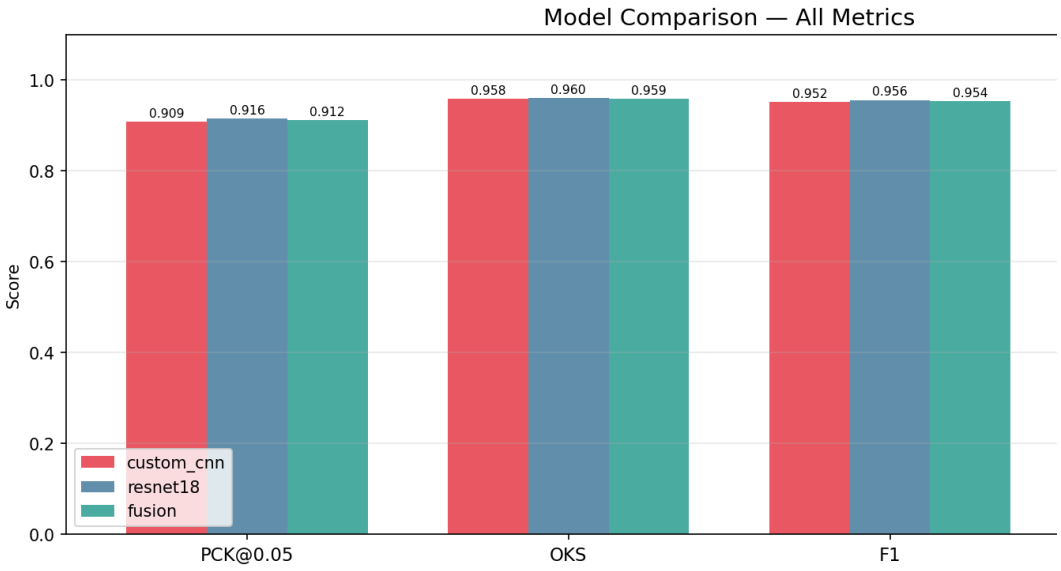

Figure 3: Final performance results of all three models in terms of PCK@0.05, OKS and F1. All three models performed more or less equally well.

begins treating them similarly anyway. In any event, the added complexity of the customCNN and fusionModel didn't buy us much, and ResNet18 is the better choice.